

1. Introduction
2. Selection statements
 if, if-else, switch
3. iterative statements
 while, do-while, for, foreach
4. transfer statements
 break, continue, return, [dowhile vs continue]

Flow control statements

=====

It decides the order in which the control will be trasfered.

flow control

=====

a. selective statement

1. if
2. if else
3. switch

b. iterative statement

1. while
2. dowhile
3. for
4. foreach

c. transfer staements

- a. break
- b. continue
- c. return
- d. System.exit(0)

if block

=====

To select some statements for execution based on condition, we use if block

syntax::

```
if(boolean)
{
    //statement-1
    //statement-2
    //statement-3
}
statement-4
```

workflow

=====

boolean condition true means statement-1,2,3 will be executed otherwise statement-4 will be executed.

eg#1.

```
int x=0;
if(x){
    System.out.println(x);
}
output:Compile time error
```

eg#2

```
int x=0;
if(x==0){
```

```
    System.out.println(x);
}
output:: 0
```

eg#3

```
boolean res=false;
if(res){
    System.out.println(res);
}
output:: nothing will be printed
```

eg#4

```
boolean res=false;
if(res=true){
    System.out.println(res);
}
output:: true
```

eg#5.

```
if (true)
System.out.println("hello");//dependent on if
System.out.println("hiee");//independent on if
```

valid cases

- a. if(true);
- b. if(false);
- c. if(true) System.out.println("hello");

invalid cases

- a. if(true) int x =10;
- b. if(true){ int x =10;} System.out.println(x);

Note:

1. If we are writing only one statement inside if block, then curly braces are optional.
2. When we write if,else block is also optional.
3. only one statement inside if block,curly braces are optional and that one statement should never be a declarative statement.
4. If we are writing declarative statement inside if block with curly braces that variable scope is limited to only if block.
5. Nesting of if-else[else-if ladder] is not preferred in coding.

```
if()
{

}
else if()
{

}
else if()
{

}
else if()
{

}
else
```

```
{  
}
```

Solution : switch case.

```
++++++  
switch  
++++++
```

if several options are available then it is not recommended to use if-else ladder we should go for "switch" statement.

switch statement improves the readability of the code.

syntax:

```
switch(x)  
{  
    case 1: action1  
    case 2: action2  
    case 3: action3  
        ;;;  
    default: default action;  
  
}
```

Until JDK1.4 version the allowed types for switch argument are

"byte, short, char, int"

From JDK1.5 version onwards Wrapper classes are also possible

"Byte, Short, Character, Integer" and enum.

From JDK1.7 version onwards we can also give "String".

Conclusions::

1. Curly braces are mandatory in switch statement. *{ } necessary*
2. If we are writing any statement inside switch then those statements should be a part of case or default.
3. If we are not writing any statement inside switch then case, default is optional.

eg#1.

```
int x= 10;  
switch (x){  
    System.out.println("hiee");  
}
```

Output: CE

eg#2.

```
int x= 10;  
int y= 20;  
switch (x){ //byte, short, int, char  
    case 10: System.out.println(10);  
    case y: System.out.println(20); //CE: constant required  
}
```

eg#3. final -> This keyword makes the variable compile time constant, so compiler will be knowing the value of the variable.

```
int x= 10;  
final int y= 20;  
switch (x){ //byte, short, int, char  
    case 10: System.out.println(10);  
    case y: System.out.println(20);
```

```
}
```

Output: Code will be compiled

Note: In java language, the memory for the variable will be given at the runtime, so we say java language as "Dynamic programming language".

eg#4. switch argument and case label can have expression, but case label should be constant expression.

```
int x= 10;
switch (x+1){
    case 10:
    case 10+20:
    case 10+20+30:
}
```

Output: No output

eg#5. case label values should lie in the range of switch argument type, otherwise it results in "Compiletime Error".

```
byte b= 10;
switch (b){//byte = -128 + 127
    case 10:System.out.println(10);
    case 100:System.out.println(100);
    case 1000:System.out.println(1000);//CE: not within the range of byte
}
```

eg#6.

```
byte b= 10;
switch (b+1){//byte + int => int
    case 10:System.out.println(10);
    case 100:System.out.println(100);
    case 1000:System.out.println(1000);
}
```

Output: No Output

eg#7.

```
int x = 10;
switch (x){ //byte---int, short---int, char('a')---int(97), int
    case 97:System.out.println("97");
    case 99:System.out.println("99");
    case 'a':System.out.println("100");
    //case 97:System.out.println("100");
}
```

Output: CE: duplicate case labels are not allowed.

Conclusion

- Case label should be compile time constant.
- Case label can have expression, but it should be a compile time constant expression.
- Case label value should be within the range of the switch argument type.
- Duplicate case label are not allowed.

+++++

Fall Through in Switch

+++++

Within a switch, if any case is matched from that case onwards all the statements will be executed until the end of switch or break.

This is called as "FallThrough" inside the switch.

The main advantage of fall through inside switch is we can define common actions for multiple cases.

```
int x = 0;
switch (x){
    case 0: System.out.println("0");
    case 1: System.out.println("1");
        break;
    case 2: System.out.println("2");
    default: System.out.println("default");
}
```

Output

```
x= 0
0
1
```

```
x=1
1
```

```
x= 2
2
default
```

```
x= 3
default
```

+++++

Duplicate case

+++++

1. within switch we can take default only once.
2. if no cases are matched then only default case will be executed.
3. With in a switch, we can take default anywhere, but it is a convention to take default as last case.

eg::

```
int x = 3;
switch (x){
    default: System.out.println("default");
    case 0: System.out.println("0");
    case 1: System.out.println("1");
        break;
    case 2: System.out.println("2");
}
```

Output

```
x=3
default
0
1
```

eg::

```
int x = 3;
switch (x){
    default: System.out.println("default");
    default: System.out.println("default");
}
```

Output: CE: duplicate default label.

+++++

Iterative statements

+++++

while loop

=> If we are not aware of how many time to iterate, then we need to go for while loop.

syntax:

```
while(boolean){
    ; ; ; ; ; ; ; ;
}
```

eg::

```
while(resultSet.next()){
    ; ; ; ;
    ; ; ; ;
}
```

```
while(enumeration.hasMoreElements()){
    ; ; ; ;
    ; ; ; ;
}
```

```
while(itr.hasNext()){
    ; ; ; ;
    ; ; ; ;
}
```

Note: The argument to a while statement should be of "boolean type".if we are using anyother type it results in "CE".

eg#1.

```
while (1)
{
    System.out.println("hello");//CE
}
```

eg#2.

```
while (true)
{
    System.out.println("hello");
}
output: hello(infinite times)
```

eg#3.

```
while (true)
;
```

Output: no output(valid)

eg#4.

```
while (true)
int x= 10; //CE: delcaration not allowed
```

eg#5.

```
while (true)
{
    int x =10; //valid
}
```

```
eg#6.
while (true)
{
    int x =10;
}
System.out.println(x);//CE: can't find symbol.
```

unreachable code -

+++++

```
eg#1.
while (true)
{
    System.out.println("hello");
}
    System.out.println("hiee");//CE: unreachable code
```

```
eg#2
while (false)
{
    System.out.println("hello");//CE: unreachable code
}
    System.out.println("hiee");
```

```
eg#3.
int a= 10;
int b= 20;

while (a<b)//JVM :: 10<20 -> true
{
    System.out.println("hello");
}
    System.out.println("hiee");
```

Output: hello(infinite times)

```
eg#4.
final int a= 10;
final int b= 20;

while (a<b)//JVM :: 10<20 -> true
{
    System.out.println("hello");
}
    System.out.println("hiee");//unreachable code
```

Output: CE

```
eg#5.
final int a= 10;
    while (a<20)
    {
        System.out.println("hello");
    }
    System.out.println("hiee");//unreachable code
```

Output: CE

eg#6.

```
int a= 10;
    while (a<20)
    {
        System.out.println("hello");
    }
    System.out.println("hiee");
```

Output: hello(infinite times)

Note::

=> Every final variable will be replaced with corresponding value by the compiler
=> if any operation involves only constants then compiler is responsible to perform operation.
=> if any operation involves at least one variable, then compiler won't perform any operation, jvm will perform that operation.

+++++

do-while loop

+++++

if we want to execute the body of the loop atleast once then we need to go for do-while loop.

Syntax:

```
do{
    ;;;;
    ;;;;
    ;;;;
}while(boolean); // ; is mandatory
```

Conclusion

- curly braces are optional
- Without curly braces we can take only one statement and that statement should never be declarative statement.

eg::

```
do{
    System.out.println("hello");
}while(true);
```

output:: hello infinite times

eg::

```
do;while(true);
output:: compiles succesfull
```

eg::

```
do
    int x=10;
while(true);
output:: compile time error
```

eg::

```
do{
    int x=10;
}while(true);
output:: compilation succesfull
```

eg::

```
dowhile(true)
```

```
System.out.println("hello");
while(true);
hello infinite times
```

eg::
 dowhile(true);
output:: compilation error.

```
eg::
do{
    System.out.println("sachin");
}while(true);
System.out.println("dhoni");//CE: unreachable code
```

```
eg::
do{
    System.out.println("sachin");
}while(false);
System.out.println("dhoni");
```

Output
sachin
dhoni

```
eg::
int a=10,b=20;
do{
    System.out.println("sachin");
}while(a<b);
System.out.println("dhoni");
```

Output: sachin(infinite times)

```
eg::
int a=10,b=20;
do{
    System.out.println("sachin");
}while(a>b);
System.out.println("dhoni");
```

output: sachin dhoni

```
eg::
final int a=10,b=20;
do{
    System.out.println("sachin");
}while(a<b);
System.out.println("dhoni");
```

output:Ce: unreachable(dhoni)

```
eg::
final int a=10,b=20;
do{
    System.out.println("sachin");
```

```
}while(a>b);  
System.out.println("dhoni");
```

output: sachin dhoni

```
eg::  
final int a=10,b=20;  
do{  
    System.out.println("sachin");  
}while(a<b);  
System.out.println("dhoni"); //unreachable
```

Output: CE

```
eg::  
final int a=10;  
do{  
    System.out.println("sachin");  
}while(a>20);  
System.out.println("dhoni");
```

Output: sachin dhoni

forloop
=====

It is one of the most commonly used loop and best suitable if we know the no of iterations in advanced.

Syntax::

```
for(initialisation;condition;incr/dec)  
{  
    //body of loop  
}
```

Note: curly braces are optional, we can take only one statement and that statement should not be declarative.

```
eg::  
int i=0;  
for(System.out.println("hello");i<3;System.out.println("hi")){  
    i++; // i = 1,2,3  
}
```

Output:: hello hi hi hi

eg:: //by default condition is kept as true by compiler

```
for(;;){  
    System.out.println("hello");  
}
```

Output:: hello(infinite times)

```
eg::  
for(;;)  
    int x=10;  
output:: CE
```

```
eg::
for(;;){
    int x=10;
}
output:: no output
```

```
eg::
for(int i=0;i<true;i++){
    System.out.println("sachin");
}
System.out.println("dhoni");
```

output::CE

```
eg::
for(int i=0;i<false;i++){
    System.out.println("sachin");
}
System.out.println("dhoni");
output::CE
```

```
eg::
for(int i=0;;i++){
    System.out.println("sachin");
}
System.out.println("dhoni");//unreachable
```

Output::CE

```
eg::
int a=10,b=20;
for(int i=0;a<b;i++){
    System.out.println("sachin");
}
System.out.println("dhoni");
output:: sachin(infinite)
```

```
eg::
final int a=10
final int b=20;
for(int i=0;a<b;i++){
    System.out.println("sachin");
}
System.out.println("dhoni");//Unreachable code
output:: CE
```

++++++
Snippets
++++++

```
Q>
Given:
int x = 0;
int y = 10;
do {
    y--; // y = y-1
    ++x; // x = x+1
} while (x < 5);
```

```
System.out.print(x + "," + y);
```

What is the result?

- A. 5,6
- B. 5,5[Answer]
- C. 6,5
- D. 6,6

```
y = 9, x = 1 :: while(1<5) true
y = 8, x = 2 :: while(2<5) true
y = 7, x = 3 :: while(3<5) true
y = 6, x = 4 :: while(4<5) true
y = 5, x = 5 :: while(5<5) false
```

Q>

```
public static void main(String[] args) {
    for (int i = 0; i <= 10; i++) {
        if (i > 6) break;
    }
    System.out.println(i);
}
```

What is the result?

- A. 6
- B. 7
- C. 10
- D. 11
- E. Compilation fails.[Answer: i not accesible outside for loop]
- F. An exception is thrown at runtime.

Q>

Consider below code of Test.java file:

```
public class Test {
    public static void main(String[] args) {
        boolean b1 = 0;
        boolean b2 = 1;
        System.out.println(b1 + b2);
    }
}
```

What is the result of compiling and executing Test class?

- A. 0
- B. 1
- C. true
- D. false
- E. compilation error[Answer: boolean means only true,false]

Q>

Given:

```
float pi = 3.141f;
if (pi > 3) {
    System.out.print("pi is bigger than 3. ");
}else {
    System.out.print("pi is not bigger than 3. ");
}finally {
    System.out.println("Have a nice day.");
}
```

What is the result?

- A. Compilation fails.[Answer:finally without try]

- B. pi is bigger than 3.
- C. An exception occurs at runtime.
- D. pi is bigger than 3. Have a nice day.
- E. pi is not bigger than 3. Have a nice day.

Q>

Given:

```
public static void main(String[] args) {  
int i = 3; //line-12  
switch(i) { //line-13  
    case 3: System.out.println("three"); break;  
    default: System.out.println("other"); break; //line-15  
}  
}
```

What is the result?

- A. three[Answer]
- B. other
- C. An exception is thrown at runtime.
- D. Compilation fails because of an error on line 12.
- E. Compilation fails because of an error on line 13.
- F. Compilation fails because of an error on line 15.

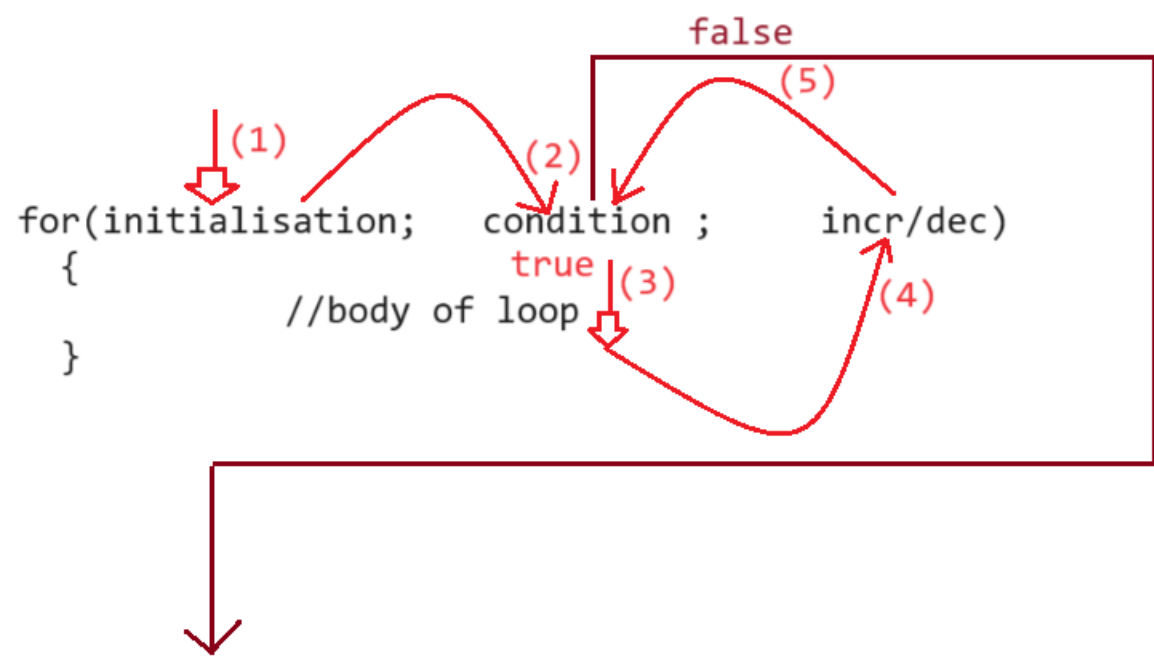
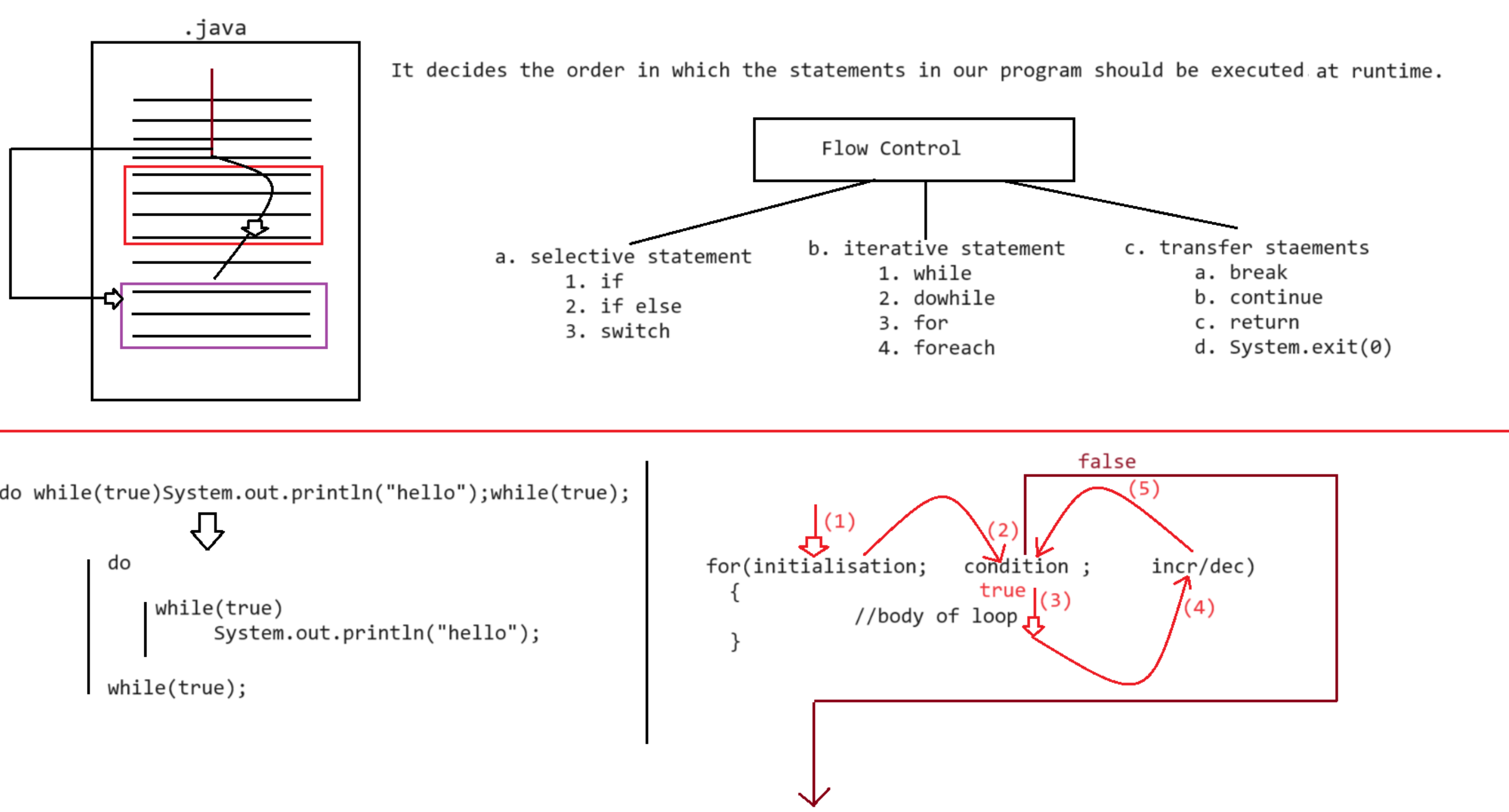
Q>

Given:

```
public static void main(String[] args) {  
boolean i = true; //line-12  
switch(i) { //line-13  
    case true: System.out.println("true"); break;  
    case false: System.out.println("false"); break;  
    default: System.out.println("other"); break; //line-15  
}  
}
```

What is the result?

- A. true
- B. false
- C. An exception is thrown at runtime.
- D. Compilation fails because of an error on line 12.
- E. Compilation fails because of an error on line 13.[Answer: switch arg types can be : byte,short,int,char]
- F. Compilation fails because of an error on line 15.



+++++

Flow Control statement

+++++

1. Simple control statements

- a. if
- b. if-else
- c. switch(tricky in java)

2. Looping control statements

- a. while[not aware of how many times iterations executes]
- b. dowhile[Atleast once if we want to execute]
- c. for[iterations if it is known at the begining]
- d. foreach(jdk1.5v)[Arrays and oops]

3. Transfer control statements

- a. break
- b. continue
- c. return[in methods]
- d. System.exit(0)[in methods]

+++++

Transfer control statements

+++++

Break statement

- a. Inside switch to avoid fallthrough
- b. Inside loops to break the loop based on some condition
- c. Inside label block to break label block execution based on some condition.

1. break is used to avoid "fallthrough" in switch.

```
int x= 0;
switch(x){
    case 0: System.out.println("hello");
        break;
    case 1: System.out.println("hiee");
}
```

2. we can use break inside loop to break the loop based on some condition.

```
for (int i=0;i<=10 ;i++ )
{
    if (i==5)
        break;
    System.out.println(i);//0 1 2 3 4
}
```

3. break can be used along with blocks also.

```
int x= 10;
l1:{
    System.out.println("begin");
    if (x==10)
        break l1;
    System.out.println("end");
}
System.out.println("hello");
```

Output

begin
hello

Note: break can be used inside "switch or loop", any other places if we try to use it results in "CompileTime-Error."

```
int x= 10;
System.out.println("hello");
if (x==10)
    break;
System.out.println("hiee");
```

```
+++++++
continue
+++++++
```

=> we can use continue statement to skip the current iteration and continue for next iteration

```
eg#1.
int x= 2;
for (int i=0;i<10 ;i++ )
{
    if (i%x==0)
        continue;
    System.out.println(i);
}
```

Output
1 3 5 7 9

eg#2. we can use continue only inside loops, if we try to use outside it would result in "CompileTime Error".

```
int x= 10;
if (x==10)
{
    continue;
}
System.out.println("hello");
```

```
do-while with continue
+++++++
int x= 0;
do
{
    x=x+1;
    System.out.println(x);

    if((x=x+1) < 5)
        continue;
    x=x+1;
    System.out.println(x);
}while (++x<10);
```

output
1
4
6
8
10

Note: Compiler won't check for unreachability in case of "if-else" statement, whereas it checks for unreachability in case of loops.

eg#1.

```
while(true)
    System.out.println("hello");
System.out.println("hiee");//CE: unreachable statement
```

```
eg#2.
if (true)
    System.out.println("hello");//hello
else
    System.out.println("hiee");
```

++++++Operators++++++

1. Increment and decrement operator
2. Arithmetic operator
3. Relational operator
4. Equality operator
5. instanceof operator
6. bitwise operator
7. shortcircuit operator
8. typecast operator
9. assignment operator
- 10.conditional operator
11. [] operator
12. precedence of operator
13. evaluation of java operands

++++++
Increment and decrement operator
++++++

increment operator

- a. pre-increment[first increment and then use it]

```
eg: int x = 10;
    int y = ++x; // x = 11, y = 11
```

- b. post-increment[first use it and then increment it]

```
eg: int x = 10;
    int y = x++; // x= 11 , y = 10
```

Note:

1. increment or decrement can be applied only on variables, but not on values directly.

```
eg: int y = 10++; //CE
```

2. we can't perform nesting of increment or decrement operator, it would result in compile time error.

```
int x= 10;
int y= ++(++x);//CE
System.out.println("x = " + x);
System.out.println("y = " + y);
```

3. For a final variable, increment or decrement operation can't be done.

```
final int x= 4;
x++;//CE
System.out.println(x);
```

4. we can't apply increment or decrement operator on boolean type, where as it can be applied on other primitive types.

```
int x= 10;
```

```

x++;
System.out.println(x);//11

char ch='a';
ch++;
System.out.println(ch);//b

double d = 10.5;
d++;
System.out.println(d);//11.5

boolean b= true;
b++;//CE
System.out.println(b);

```

5. What is the difference b/w `b = b+1` and `b++`?

```

byte b= 10;
b = b+1; //CE (byte+int :: int)
System.out.println(b);

```

```

byte b = 10;
b++; // b = (byte)(b+1);
System.out.println(b);//11

```

```

+++++
Arithmetic operator
+++++
operators : +, -, *, /, %

```

Note: if we apply arithmetic operators b/w 2 variables then the result type is always `max(int,typeof a, typeof b)`

```

eg: byte + byte    = int
    byte + short   = int
    int + double   = double
    char + char    = int
    char + double  = double

```

eg#1.

```

System.out.println('a' + 'b');//195
System.out.println('a' + 1);// 98
System.out.println('a' + 1.2);//98.2

```

eg#2. In integral arithmetic(byte,short,int,long) there is no way to represent "Infinity", so result will be "ArithmeticException".

In case of double,float types, there is a possibility of storing "Infinity" so the result will be "Infinity".

```

System.out.println(10/0.0); // int/double = +double
System.out.println(-10/0.0); // -int/double = -double
System.out.println(0/0); // int/int = int

```

output

Infinity

-Infinity

ArithmeticException :/by zero

++++++
Snippets
++++++

What will be the result of compiling and executing Test class?

```
public class Test {  
    public static void main(String[] args) {  
        boolean b = true;  
        switch(b) { //switch arg are : byte, short, int, char, String, enum  
            case true:  
                System.out.println("ONE");  
            case false:  
                System.out.println("TWO");  
            default:  
                System.out.println("THREE");  
        }  
    }  
}
```

A. ONE
TWO
THREE

B. TWO
THREE

C. THREE

D. None of the above options

Answer: D

```
Q>  
public static void main(String[] args) {  
    for (int i = 0; i <= 10; i++) {  
        if (i > 6) break;  
    }  
    System.out.println(i); //CE: undefined variable i  
}
```

What is the result?

A. 6
B. 7
C. 10
D. 11
E. Compilation fails.
F. An exception is thrown at runtime.

Answer: E

Q>

Given code of Test.java file:

```
public class Test {  
    public static void main(String[] args) {  
        byte b1 = 10; //Line n1  
        int i1 = b1; //Line n2  
        byte b2 = i1; //Line n3[implicit type casting is not possible]  
        System.out.println(b1 + i1 + b2);  
    }  
}
```

What is the result of compiling and executing Test class?

- A. Line n1 causes compilation error
- B. Line n2 causes compilation error.
- C. Line n3 causes compilation error.
- D. 30 printed on to console.

Answer: C

Q>

What will be the result of compiling and executing the Test class?

```
public class Test {  
    public static void main(String[] args) {  
        int grade = 75;  
        if(grade > 60)  
            System.out.println("Congratulations");  
        System.out.println("You passed");  
        else  
            System.out.println("You failed");  
    }  
}
```

- A. Congratulations
- B. Congratulations
You passed
- C. You failed
- D. compilation error[misplaced else]

Q>

What will be the output of compiling and executing the Test class?

```
public class Test {  
    public static void main(String[] args) {  
        int x = 2;  
        switch (x) {  
            default:  
                System.out.println("Still no idea what x is");  
            case 1:  
                System.out.println("x is equal to 1");  
                break;  
            case 2:  
                System.out.println("x is equal to 2");  
                break;  
            case 3:  
                System.out.println("x is equal to 3");  
                break;  
        }  
    }  
}
```

- A. x is equal to 2[Answer]
- B. Compilation error
- C. Still no idea what x is
- D. Produces nooutput


```
11:for(;;)
{
12:for(;;)
{
13:for(;;)
{ break 11;
break 13;
break;
break 12;
}
}
}
```

```
for (int i=0;i<3; i++)
{
for (int j=0;j<3 ;j++ )
{
if(i==j)
break;
System.out.println(i+" .... " + j);
1      ....   0
2      ....   0
2      ....   1
}
```

i = 0(Outer loop) :0<3(true)	i = 1(Outer loop): 1<3(true)	i=2(Outer loop): 2<3(true)	i=3(Outer loop) :3<3(false)
j = 0(Inner loop) :0<3(true)	j=0(Inner loop): 0<3(true)	j=0(Inner loop): 0<3(true)	
0==0(true)	1==0 (false)	2==0(false)	
	1==1 (true)	2==1(false)	
		2==2(true)	

```
11: for (int i=0;i<3; i++)
{
for (int j=0;j<3 ;j++ )
{
if(i==j)
break 11;
System.out.println(i+" .... " + j);
}
}
```

```
for (int i=0;i<3; i++)
{
for (int j=0;j<3 ;j++ )
{
if(i==j)
continue;
System.out.println(i+" .... " + j);
}
}
```

i = 0 (Outer loop) :0<3(true)	i = 1(Outer loop) :1<3(true)	i = 2(Outer loop) : 2<3(true)
j = 0(Inner loop) :0<3(true)	j = 0(Inner loop) :0<3(true)	j = 0(Inner loop) : 0<3(true)
0==0(true)	1==0(false)	2==0(false)
0==1(false) 1<3(true)	1==1(true)	2==1(false)
0==2(false) 2<3 (true)	1==2(false) :3<3(false)	2==2(true) : 3<3(false)
3<3(false)		

```
11: for (int i=0;i<3; i++)
{
for (int j=0;j<3 ;j++ )
{
if(i==j)
continue 11;
System.out.println(i+" .... " + j);
}
}
```

i=0(Outerloop) :0<3(true)	i=1(Outerloop) :1<3(true)	i=2(Outerloop) : 2<3(true)	i=3(Outerloop): 3<3(false)
j=0(Innerloop) :0<3(true)	j=0(Innerloop) :0<3(true)	j=0(Innerloop) : 0<3(true)	
0==0	1==0(false)	2==0(false)	
	1==1(true)	2==1(false)	
		2==2(true)	

```
int x= 0;
do
{
x=x+1;
System.out.println(x);
if((x==x+1) < 5)
continue;
x=x+1;
System.out.println(x);
}while (++x<10);
```

x = 0	x =3 : 3<10(true)	x = 7 : 7<10(true)
x = 1	x = 4	x = 8
x = 2 : 2<5(true)	x = 5 : 5<5(false)	x= 9 :9<5(false)
	x = 6	x= 10
		x = 11: 11<10(false)

```
int x = 10;
int y = ++x;
```

x = 10 11	.
y = 11	Pre-increment

```
int x = 10;
int y = x++;
```

x = 10 11	.
y = 10	Post-increment